كلية الهندسة بشبرا

FACULTY OF ENGINEERING AT SHOUBRA

Computer Organization

Bank System Project

Group : 6

Islam Ahmed

Gehad Amr

Nada Ashraf

Nesreen Ahmed

Nour Abdel Sater

Dr. LAMIA.ALREFAAI

2025

Table of Contents

Abstract	**1**
Introduction	**1**
System Overview	**1**
Code Structure and Functions	**2**
Code Explanation	**4**
Results	**14**
Conclusion	**18**

Abstract

This report presents the design and implementation of a simplified Bank Management System using MIPS assembly language. The system simulates core banking operations such as account registration, login authentication, balance inquiry, deposit, withdrawal, and fund transfer. Developed on the MIPS32 architecture, this project demonstrates how low-level programming techniques can be applied to manage structured data and control flow in a banking environment. The report provides a detailed explanation of the system's design, implementation logic, and testing procedures.

Introduction

Banking systems are critical infrastructures that require precision, efficiency, and robust logic to manage financial transactions securely. While these systems are typically implemented using high-level programming languages and complex database technologies, understanding their core functionalities at a low level offers valuable insights into how computers handle data, execute control flow, and manage memory.

This project focuses on designing and implementing a simplified Bank Management System using **MIPS assembly language**, a low-level programming language commonly used in computer architecture education. By simulating fundamental banking operations—such as account registration, user authentication, balance inquiry, deposits, withdrawals, and fund transfers—this project illustrates how a micro-level system can replicate the essential behavior of real-world banking software.

The goal of this implementation is not only to replicate banking functionality but also to deepen understanding of assembly-level programming, system calls, memory manipulation, and control structures. The use of a MIPS simulator such as QtSPIM provides a platform for executing and debugging the system in a controlled environment.

Through this project, students and developers gain practical experience in handling user input, comparing strings, managing structured data, and controlling program logic—all within the constraints and capabilities of the MIPS architecture.

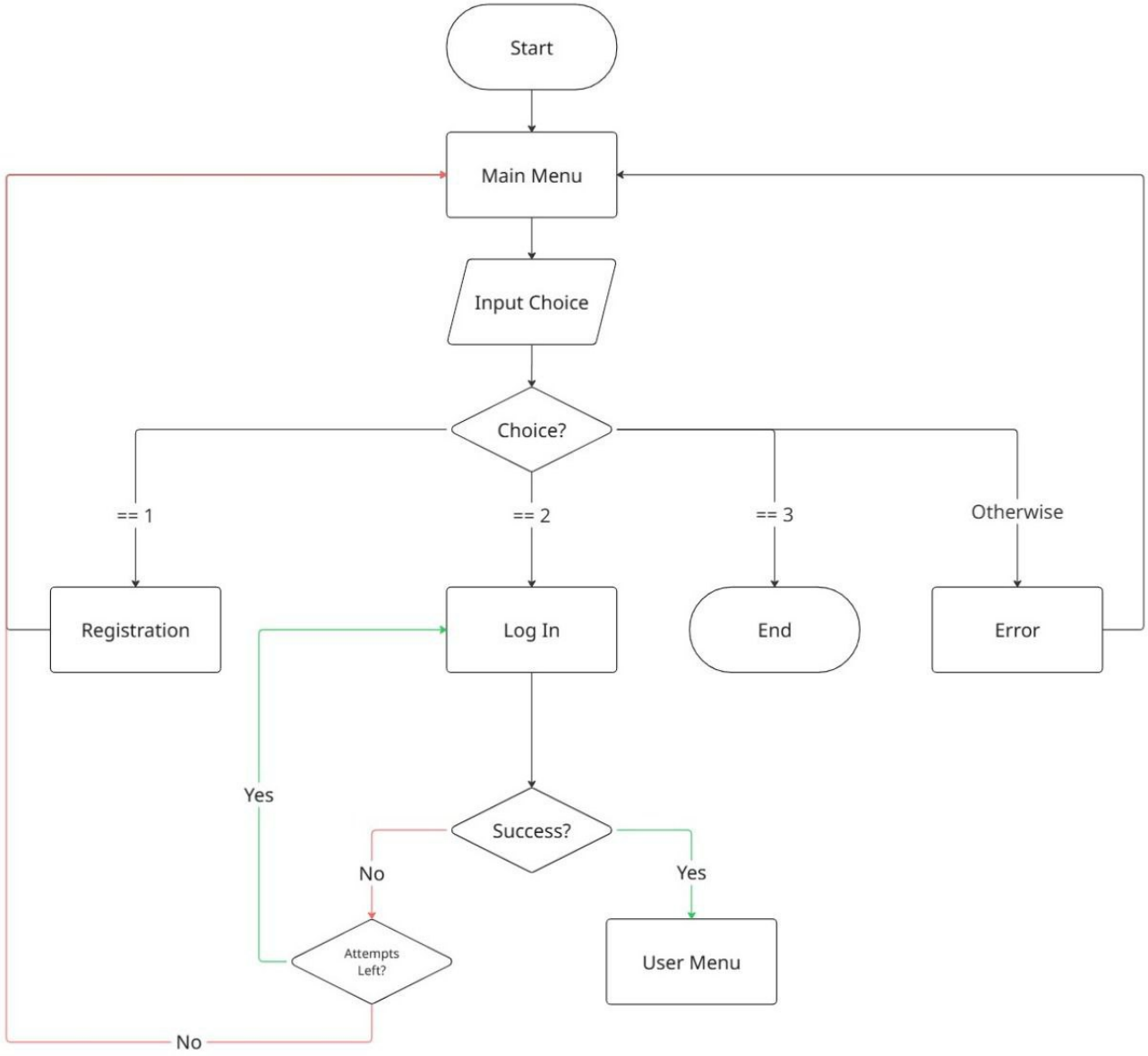
System Overview

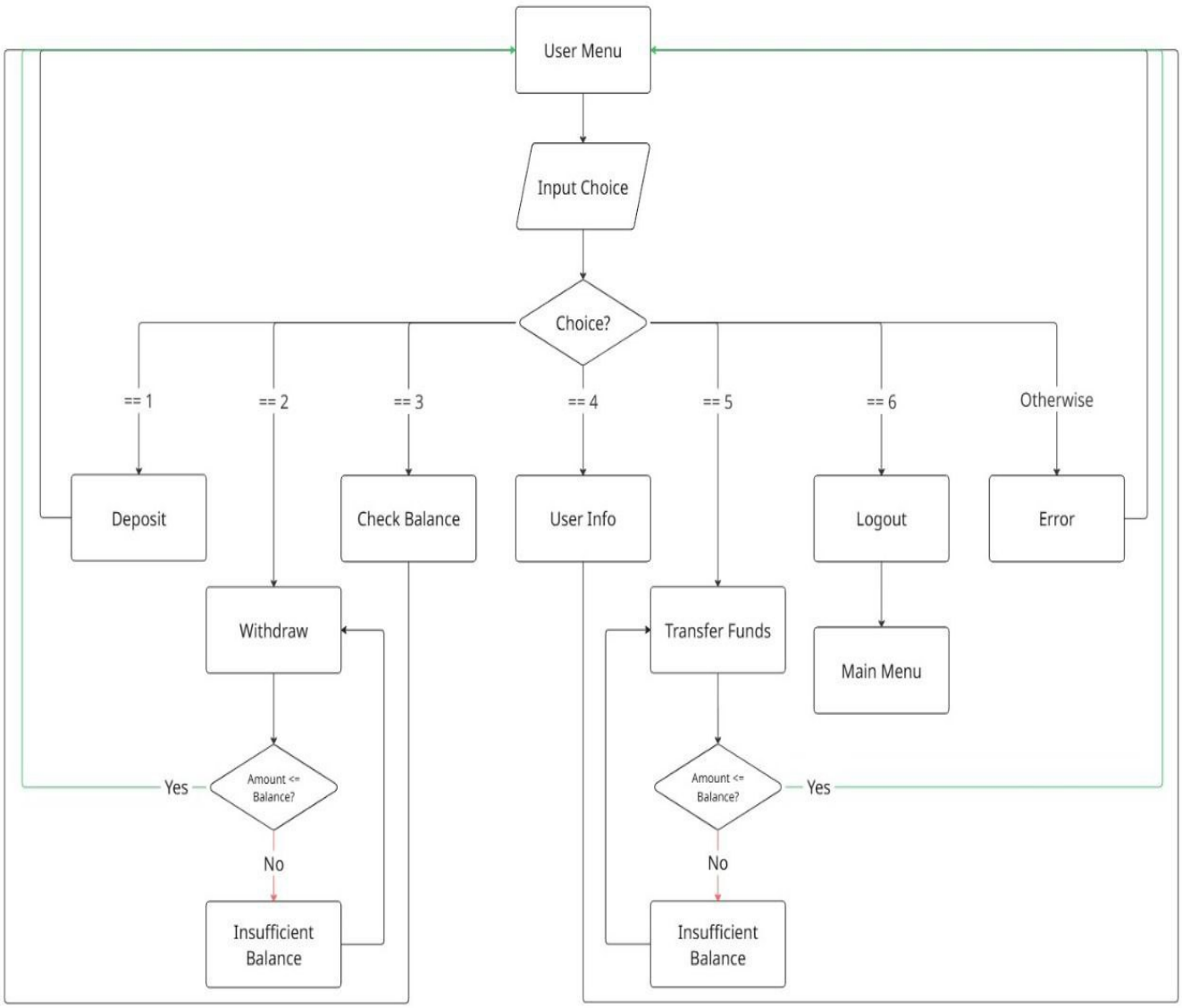
The Bank Management System supports the following functionalities:

- Account registration
- Login and authentication
- Balance inquiry
- Deposit and withdrawal operations
- Fund transfers between accounts
- Account information display

These operations are implemented using basic MIPS system calls, conditional branches, memory operations, and looping constructs.

Code Structure and Functions




User Menu
Input Choice
Choice?
== 1
== 2
== 3
== 4
== 5
== 6
Otherwise
Deposit
Check Balance
User Info
Logout
Error
Withdraw
Transfer Funds
Main Menu
Amount <= Balance?
Yes
No
Insufficient Balance
Amount <= Balance?
Yes
No
Insufficient Balance

Code Explanation

I. .data section :

Step 1: Main Menu Display

- `prompt1` – `prompt7` : Show welcome screen and options (Register, Login, Exit).

Step 2: Registration Prompts

- `reg1` – `reg3` : Section header for registration.
- `accName` , `accAddress` , `accCountactNum` , `userName` , `pass` , `depAmount` : Ask for personal info, username, password, and deposit.

Step 3: Decorative Lines

- `astreik` , `astreik1` , `astreik2` : Display lines of asterisks for formatting.

Step 4: Post-Login Menu

- `login1` – `login7` : Options after login (Deposit, Withdraw, Balance, Info, Transfer, Logout).

Step 5: Login Prompts

- `EnteruserName` , `Enterpass` : Ask for login credentials.
- `invlaied` , `setAgain` : Show invalid password message and retry.

Step 6: Banking Operations

- `Enter` : Ask for amount.
- `msgDeposit` , `msgWithdraw` , `msgTransfer` : Confirm actions.
- `insuffitent` : Warn if balance is low.
- `currentBlance` : Show balance.
- `transferPrompt` : Ask for target account username.

Step 7: Error/Exit Messages

- `exit1` – `exit6` : Error screen if something fails or user exits.

Step 8: Show User Info

- `info1` – `info3` : Print name, address, and contact number.

Step 9: End of Program

- `end1` : Thank you message.
- `incorrext` : Shown if login or input fails.
- `chances` , `chances1` : Show remaining attempts.

Step 10: Variables (Memory)

- `Name`, `Address`, `Contact`: Store user personal info.
- `storedUser`, `storedPass`: Save registered credentials.
- `inputUser`, `inputPass`: Store user login input.
- `targetUser`: Store the username of the transfer receiver.

II. .text section :

1. ### OPTION_DISPLAYING_OUTER

```
option_Displaying_Outer:
   li $s5, 2       # Initialize number of login attempts to 2
   li $v0, 4
   la $a0, prompt1
   syscall
   li $v0, 4
   la $a0, prompt2
   syscall
   li $v0, 4
   la $a0, prompt3
   syscall
   li $v0, 4
   la $a0, prompt4
   syscall
   li $v0, 4
   la $a0, prompt5
   syscall
   li $v0, 4
   la $a0, prompt6
   syscall
   j choice
```

Explanation

1. **Set login attempts:**
   - `li $s5, 2`
     Initialize register `$s5` to 2, representing the allowed login attempts.
2. **Display menu header line 1:**
   - Load and print the string at label `prompt1` (decorative line).
3. **Display menu header line 2:**
   - Load and print the string at label `prompt2` ("BANK OF GINNN" title).

4. **Display menu header line 3:**
   - Load and print the string at label `prompt3` (decorative line).
5. **Display menu option 1:**
   - Load and print the string at label `prompt4` ("1. Register account.").
6. **Display menu option 2:**
   - Load and print the string at label `prompt5` ("2. Login.").
7. **Display menu option 3:**
   - Load and print the string at label `prompt6` ("3. Exit.").
8. **Jump to user input handling:**
   - Jump to the label `choice` where the program waits for user input.

2. ### USER CHOICE HANDLER

```
choice:
   li $v0, 4
   la $a0, prompt7       # "Enter your choice"
   syscall
   li $v0, 5
   syscall
   move $t0, $v0         # Store user input in $t0
   beq $t0, 1, acountRegisteration   # Go to account registration
   beq $t0, 2, loginSystem           # Go to login
   beq $t0, 3, end                   # Exit
   jal exitHandler                   # Invalid option handler
   j option_Displaying_Outer
```

Explanation

1. Display the prompt "Enter your choice".
2. Read the user's numeric input.
3. Store the input in register `$t0`.
4. If input = 1, jump to account registration.
5. If input = 2, jump to login system.
6. If input = 3, jump to program end.
7. If input is invalid (not 1, 2, or 3), call the exit handler and then redisplay the main menu.

3. ### ACCOUNT REGISTRATION

```
acountRegisteration:
    la $a0, reg1
    li $v0, 4
    syscall
    la $a0, reg2
    li $v0, 4
    syscall
    la $a0, reg3
    li $v0, 4
    syscall
    # Name input
    la $a0, accName
    li $v0, 4
    syscall
    li $v0, 8
    la $a0, Name
    li $a1, 40
    syscall
    # Address input
    la $a0, accAdd
    li $v0, 4
    syscall
    li $v0, 8
    la $a0, Address
    li $a1, 40
    syscall
    # Contact input
    la $a0, accContact
    li $v0, 4
    syscall
    li $v0, 8
    la $a0, Contact
    li $a1, 15
    syscall
    # Username
    la $a0, reg4
    li $v0, 4
    syscall
    li $v0, 8
    la $a0, storedUser
    li $a1, 20
    syscall
    # Password
    la $a0, reg5
    li $v0, 4
    syscall
    li $v0, 8
    la $a0, storedPass
    li $a1, 20
    syscall
    # Initial balance
    la $a0, reg6
```

```
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    move $t6, $v0      # Save balance
    j option_Displaying_Outer
```

Explanation

1. Print registration header messages.
2. Prompt and read user name (string).
3. Prompt and read address (string).
4. Prompt and read contact number (string).
5. Prompt and read username (string).
6. Prompt and read password (string).
7. Prompt and read initial balance (integer).
8. Save balance and return to main menu.

4. ### LOGIN SYSTEM

```
loginSystem:
    # Username input
    la $a0, EnteruserName
    li $v0, 4
    syscall
    li $v0, 8
    la $a0, inputUser
    li $a1, 20
    syscall
PASS:
    la $a0, Enterpass
    li $v0, 4
    syscall
    li $v0, 8
    la $a0, inputPass
    li $a1, 20
    syscall
    # Compare username
    la $a0, storedUser
    la $a1, inputUser
    jal strcmp
    bnez $v0, invalid     # Invalid username
    # Compare password
    la $a0, storedPass
    la $a1, inputPass
    jal strcmp
```

```
beq $v0, $zero, underAccount  # Match → go to account
```

Explanation

1. Display prompt to enter username.
2. Read username input into temporary buffer.
3. Display prompt to enter password.
4. Read password input into temporary buffer.
5. Compare entered username with stored username using strcmp.
6. If usernames do not match, jump to invalid handler.
7. If usernames match, compare entered password with stored password using strcmp.
8. If passwords match, jump to underaccount (login success).
9. Otherwise, handle invalid login.

5. ### INVALID LOGIN

```
invalid:
   la $a0, invalidMsg
   li $v0, 4
   syscall
   subi $s5, $s5, 1       # Decrease remaining attempts
   bnez $s5, loginSystem  # Retry if attempts left
   j exitHandler
```

Explanation

1. Display invalid login message.
2. Decrease the remaining login attempts by 1.
3. If there are attempts left, jump back to the login system to retry.
4. If no attempts remain, jump to the exit handler (end program or lockout).

6. ### LOGGED-IN ACCOUNT MENU

```
underAccount:
   li $s5, 2   # Reset login attempts
   la $a0, astreik
   li $v0, 4
   syscall
   la $a0, login1     # Welcome message
   li $v0, 4
   syscall
   la $a0, Name
   li $v0, 4
```

```
    syscall
    la $a0, astreik1
    li $v0, 4
    syscall
option_Displaying_Inner:
    li $v0, 4
    la $a0, login2   # Deposit
    syscall
    li $v0, 4
    la $a0, login3   # Withdraw
    syscall
    li $v0, 4
    la $a0, login4   # Check Balance
    syscall
    li $v0, 4
    la $a0, login5   # Show Info
    syscall
    li $v0, 4
    la $a0, login6   # Transfer Funds
    syscall
    li $v0, 4
    la $a0, login7   # Logout
    syscall
    j choice_2
```

Explanation

1. Print an asterisk line (decorative).
2. Print a welcome/login success message.
3. Print the user's name.
4. Print another asterisk line.
5. Display the inner account options menu with multiple prompts.
6. Jump to `choice_2` to get the user's option selection for account actions.

7. ### ACCOUNT OPERATIONS

```
choice_2:
    li $v0, 4
    la $a0, login8   # Enter your choice
    syscall
    li $v0, 5
    syscall
    move $t1, $v0
    beq $t1, 1, deposit
    beq $t1, 2, withdraw
    beq $t1, 3, balance
    beq $t1, 4, info
```

```
    beq $t1, 5, transfer
    beq $t1, 6, option_Displaying_Outer
    jal exitHandler
deposit:
    la $a0, depositMsg
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    add $t6, $t6, $v0   # Add amount to balance
    j option_Displaying_Inner
withdraw:
    la $a0, withdrawMsg
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    sub $t6, $t6, $v0
    j option_Displaying_Inner
balance:
    la $a0, balanceMsg
    li $v0, 4
    syscall
    move $a0, $t6
    li $v0, 1
    syscall
    j option_Displaying_Inner
info:
    la $a0, accinfoMsg
    li $v0, 4
    syscall
    la $a0, Name
    li $v0, 4
    syscall
    la $a0, Name
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, astreik1
    syscall
    la $a0, info2
    li $v0, 4
    syscall
    la $a0, Address
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, astreik1
    syscall
    li $v0, 4
    la $a0, info3
    syscall
    la $a0, Contact
    li $v0, 4
```

```
    syscall
    li $v0, 4
    la $a0, astreik1
    syscall
    j option_Displaying_Inner
transfer:
    la $a0, transMsg
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    sub $t6, $t6, $v0
    j option_Displaying_Inner
```

Explanation

1. **Prompt user to enter choice** for banking operations.
2. **Read user input** and save in `$t1`.
3. **Check choice:**
   - If 1, go to **deposit**.
   - If 2, go to **withdraw**.
   - If 3, go to **balance**.
   - If 4, go to **info**.
   - If 5, go to **transfer**.
   - If 6, return to the **outer menu**.
   - Else, call **exitHandler** for invalid choice.
4. **Deposit:**
   - Print deposit prompt.
   - Read amount.
   - Add amount to balance `$t6`.
   - Return to account menu.

5. **Withdraw:**
   - Print withdraw prompt.
   - Read amount.
   - Subtract amount from balance `$t6`.
   - Return to account menu.

6. **Balance:**
   - Print balance message.
   - Print current balance from `$t6`.
   - Return to account menu.
7. **Info:**
   - Print account information labels and stored details (Name, Address, Contact).
   - Return to account menu.
8. **Transfer:**
   - Print transfer prompt.
   - Read transfer amount.
   - Subtract amount from balance `$t6`.
   - Return to account menu.

8. ### EXIT HANDLER

```
exitHandler:
    la $a0, exitMsg
    li $v0, 4
    syscall
    j end
```

Explanation

1. Load the address of the string exitMsg into $a0.
2. Set $v0 to 4 to indicate a print string syscall.
3. Execute syscall to display the exit message.
4. Jump to the end label to terminate or exit the program.

9. ### PROGRAM END

```
end:
    li $v0, 10
    syscall
```

Explanation

1. li $v0, 10

   → Load the exit system call code (10) into register $v0.

2. syscall

   → Execute the system call to terminate the program.

Results

```
                    ====================
                        BANK OF GINNN
                    ====================

                    1. Register account.
                    2. Login.
                    3. Exit.
Enter your choice : 1

                    =======================
                    Account Registeration
                    =======================
Enter Name: Islam Asar
Enter Address: islam*****@gmail.com
Enter Contact Number: 01*********
Set Username: islam
Set Password: 123456
Enter initial amount: 1500
```

```
                  ===================
                      BANK OF GINNN
                  ===================

                  1. Register account.
                  2. Login.
                  3. Exit.
Enter your choice : 2

Enter Username: islam

Enter Password: 123456

*****************************************************************************
                           Welcome Islam Asar
*****************************************************************************
                           1. Deposit.
                           2. Withdraw.
                           3. Current Balance.
                           4. User Information.
                           5. Transfer Funds.
                           6. Log out.
Enter your choice :
```

```
*******************************************************************************
                               Welcome Islam Asar
*******************************************************************************
                               1. Deposit.
                               2. Withdraw.
                               3. Current Balance.
                               4. User Information.
                               5. Transfer Funds.
                               6. Log out.
Enter your choice : 6

                     ====================
                         BANK OF GINNN
                     ====================

                     1. Register account.
                     2. Login.
                     3. Exit.
Enter your choice :
```

```
                    ====================
                        BANK OF GINNN
                    ====================

                    1. Register account.
                    2. Login.
                    3. Exit.
Enter your choice : 2

Enter Username: islm

Enter Password: 123456

!!!!!!!!!!!!!!!Invalid Username or Password!!!!!!!!!!!!!!!
                     Note: You have 2 chances
Set again

Enter Username: islam

Enter Password: 1234

!!!!!!!!!!!!!!!Invalid Username or Password!!!!!!!!!!!!!!!
                     Note: You have 1 chances
Set again

Enter Username:

Enter Password: 123456

!!!!!!!!!!!!!!!Invalid Username or Password!!!!!!!!!!!!!!!
                     Note: You have 0 chances
*******************************************************************************
                    !!!!!!!!Incorrect!!!!!!!!
                       Return To Main Menu
*******************************************************************************
                    ====================
                        BANK OF GINNN
                    ====================

                    1. Register account.
                    2. Login.
                    3. Exit.
Enter your choice :
```

```
                              1. Deposit.
                              2. Withdraw.
                              3. Current Balance.
                              4. User Information.
                              5. Transfer Funds.
                              6. Log out.
Enter your choice : 1

Enter Amount: 500
*******************************************************************************
500 Deposited To Your Account
*******************************************************************************
                              1. Deposit.
                              2. Withdraw.
                              3. Current Balance.
                              4. User Information.
                              5. Transfer Funds.
                              6. Log out.
Enter your choice : 3

*******************************************************************************
Your current Balance is: 2000
*******************************************************************************
```

```
                                1. Deposit.
                                2. Withdraw.
                                3. Current Balance.
                                4. User Information.
                                5. Transfer Funds.
                                6. Log out.
Enter your choice : 2

Enter Amount: 150
*******************************************************************************
150 Debited From Your Account
*******************************************************************************
                                1. Deposit.
                                2. Withdraw.
                                3. Current Balance.
                                4. User Information.
                                5. Transfer Funds.
                                6. Log out.
Enter your choice : 3

*******************************************************************************
Your current Balance is: 1850
*******************************************************************************
```

```
                                1. Deposit.
                                2. Withdraw.
                                3. Current Balance.
                                4. User Information.
                                5. Transfer Funds.
                                6. Log out.
Enter your choice : 4

*******************************************************************************
Account holder Name: Islam Asar
*******************************************************************************
Account holder Address: islam*****@gmail.com
*******************************************************************************
Account holder Contact Number: 01*********
*******************************************************************************
```

```
                                1. Deposit.
                                2. Withdraw.
                                3. Current Balance.
                                4. User Information.
                                5. Transfer Funds.
                                6. Log out.
Enter your choice : 5

Enter Target Account Username: gehad

Enter Amount: 1500
*******************************************************************************
1500 Transferred To Target Account
*******************************************************************************
```

```
                                1. Deposit.
                                2. Withdraw.
                                3. Current Balance.
                                4. User Information.
                                5. Transfer Funds.
                                6. Log out.
Enter your choice : 3

*******************************************************************************
Your current Balance is: 350
*******************************************************************************
                                1. Deposit.
                                2. Withdraw.
                                3. Current Balance.
                                4. User Information.
                                5. Transfer Funds.
                                6. Log out.
Enter your choice : 2

Enter Amount: 500

*******************************************************************************
Insufficient Amount In Your Bank
*******************************************************************************
```

```
                                1. Deposit.
                                2. Withdraw.
                                3. Current Balance.
                                4. User Information.
                                5. Transfer Funds.
                                6. Log out.
Enter your choice : 5

Enter Target Account Username: gehad

Enter Amount: 500

*******************************************************************************
Insufficient Amount In Your Bank
*******************************************************************************
```

```
                    ====================
                        BANK OF GINNN
                    ====================

                    1. Register account.
                    2. Login.
                    3. Exit.
Enter your choice : 3

*******************************************************************************
                        ThankYou For Using GINNN Bank
                                Please Come Again
*******************************************************************************
-- program is finished running --
```

Conclusion

The MIPS Assembly Bank Management System project demonstrates the practical application of low-level programming in managing fundamental banking operations such as user registration, login authentication, deposit, withdrawal, balance inquiry, information display, and fund transfer. Through the use of memory allocation, string handling, and conditional branching, this project simulates a basic yet functional banking environment. It reinforces essential concepts in assembly language such as register manipulation, system calls, and memory addressing.

This project not only highlights the logical structure behind real-world banking software but also strengthens the programmer's understanding of MIPS architecture and low-level system control. By building this application from the ground up, students and developers can better appreciate the interaction between hardware and software, and develop a deeper, more technical insight into how complex systems operate at the machine level. Overall, this project serves as a strong foundation for learning system-level programming and optimizing performance-critical applications.

Thank You ....